Companies implementation plan

A four-hour build sequence, focused tests and a reviewable submission.

Implement the PRD as a small vertical feature. Make the API and typed transport correct first, then connect URL-driven queries and a single drawer. Reserve time for the two required component tests, final commands and clear documentation.

State ownership and component boundaries

Owner	Responsibility
Server Components	Layout/page shell, selected-language dictionary and server-rendered labels. Use loading.tsx for route loading and error.tsx for unexpected render failures.
Client feature	Interactive filters, table, drawer and mutations. useSearchParams reads URL state; a Suspense boundary covers the client entry where required for prerendering [1, 2].
URL	q, status, sort, page and mutually exclusive company/drawer=create. One parser produces normalized list parameters and drawer state.
TanStack Query	Company lists, detail records, requests, errors and mutation status. Keep one stable browser QueryClient; no server singleton query cache.
Form state	Draft values, original baseline/version and dirty fields. Initialize for a new edit session; never reset from every query result.
Fake auth	Client auth provider loads a demo token from sessionStorage before enabling queries. A shared token getter attaches the current token at request time.

Suggested file layout

```
app/
  layout.tsx                # server locale/dictionary
  providers.tsx             # stable QueryClient + auth
  sign-in/page.tsx
  companies/{page,loading,error}.tsx
  api/v1/companies/route.ts
  api/v1/companies/[id]/route.ts
features/companies/
  contracts.ts              # schema-derived types
  api.ts                    # four typed operations
  queries.ts                # keys + queryOptions
  url-state.ts              # parse/serialize
  companies-screen.tsx
  company-table.tsx
  company-drawer.tsx
  company-form.tsx
lib/{auth,i18n}/
server/companies-store.ts     # server-only memory array
tests/                      # unit + component tests
```

Use TanStack Query v5, Zod for shared runtime validation, Vitest, React Testing Library, user-event and MSW. Avoid adding a global state library, table framework or form framework for this small feature. These are proposed choices, not mandated tools.

Build order and time budget

Timebox the practical build to 240 minutes. Prepare and personalize Part 1 in another 35-45 minutes, keeping the full exercise near the stated four-to-five-hour budget.

Minutes	Implementation step	Exit check
0-20	**1. Scaffold.** Fresh Next.js 16 App Router project; React 19, Tailwind, strict TypeScript. Configure Query provider and test harness. Set scripts and pin versions/lockfile.	Application and one test run. lint uses eslint ., typecheck uses tsc --noEmit, test uses vitest run, build uses next build [3].
20-50	**2. API.** Create Company/input schemas, seed 40 records, shared delay/auth/error helpers and four Route Handlers.	Bearer checks; list total/filter/sort/page; create 201; update increments; stale PATCH 409; missing ID 404.
50-70	**3. Client and auth.** Four typed methods, unknown-to-schema response parsing, ApiError, token injection, abort support and query options. Fake sign-in/401 handling.	Authenticated request succeeds; 401 redirects once; error type/status is preserved; unit key/client test passes.
70-105	**4. URL list.** Implement parser, search submission, status, sorting, paging and mounted-table states. Write filtering component test.	Refresh/Back restore controls and results; limit=20; old results remain while updating; drawer params do not refetch list unnecessarily.
105-135	**5. Drawer and writes.** Shared create/detail drawer; detail query; local edit session; validation; POST/PATCH; success cache handling.	Deep link and Escape work. PATCH sends original version. Client and server validation agree; API 422 displays.
135-160	**6. Conflict recovery.** Implement 409 notice, preserved draft, explicit latest reload and reviewed resubmission. Write the 409 component test.	First PATCH v1 fails, reload obtains v2 without losing edits, explicit resubmit sends v2 and succeeds as v3.
160-180	**7. NL/EN and usability.** Selected dictionary, Intl dates, labels, error announcements and focus behavior.	Language switch keeps filter/draft state. Keyboard can open, use and close drawer; focus returns.
180-215	**8. Finish verification.** Complete component/unit assertions, API contract checks and manual browser walkthrough; run all four required commands.	Required tests and commands pass. Fix correctness failures before spending time on appearance.
215-240	**9. Package.** Final README, short answers, meaningful final commit and clean-checkout verification.	Reviewer can install/run/test and reproduce 409. Repo access is correct; no secrets, debug code or exaggerated claims.

Scope control when time is tight

Keep search on Enter or a Search button rather than adding debounce machinery. Use plain controls and a semantic table. Cut decorative UI, abstractions and optional E2E automation first; keep the specified workflows, required tests and final four commands. If anything remains incomplete, identify the exact gap in the README.

API and typed data implementation

Store and Route Handlers

Use the Node.js runtime and a server-only module with one typed process-global array so both route files share the same seed in a local process. Make the seed deterministic, with varied names, nullable domains, statuses and ISO dates. Document restart and multi-instance limitations. Do not describe this as persistent storage.

For each supported operation: await a random delay of 300 + floor(random * 501) ms; validate the bearer shape; parse/validate inputs; return a typed JSON result. In dynamic handlers await params before reading id [4]. Return no-store cache headers and do not introduce Next.js shared caching for authenticated records.

For PATCH, perform the final read, version comparison and array replacement synchronously after all awaits. That makes the compare/update atomic within this single JavaScript process. Preserve creation timestamp, increment version once, update modified timestamp, and never mutate a returned shared reference. Tests should show two writes using the same version cannot both succeed.

Typed client and errors

Expose listCompanies(params), getCompany(id), createCompany(input) and updateCompany(id, input). Central request code obtains the current token, attaches Authorization, checks response.ok, parses unknown JSON and validates it with shared schemas. Model error status and field details; never cast arbitrary JSON directly to Company. Propagate AbortSignal from query functions.

```
list key   = ['companies', 'list', {q, status, sort, limit, offset}]
detail key = ['companies', 'detail', id]
list prefix = ['companies', 'list']
```

Build keys and request parameters from the same normalized object. Expose listCompaniesOptions(params) and companyOptions(id) with queryOptions [5]. Include every input affecting the response [6]; omit tokens, locale and drawer state. In a real tenant-aware app, include non-secret organization/session scope and clear caches on account changes.

Auth and request lifecycle

Use sessionStorage only for the fake non-secret demo token so refresh works in the tab. Gate company queries until storage initialization completes. On any 401, synchronously mark auth invalid, disable queries, cancel in-flight work, clear token and Query cache, then replace to /sign-in with a validated local return path. Deduplicate simultaneous redirects. No automatic retries for 401, 404, 409 or 422; at most one query retry for network/5xx, and no automatic mutation retries.

The real-product answer uses a hosted-identity SDK token getter, renewal and production-appropriate storage. The mock deliberately has no expiry/refresh endpoint. Never put tokens in URLs, logs, cache keys, checked-in files or server-rendered props. sessionStorage is JavaScript-readable and is a demo trade-off [10].

URL updates and stable rendering

Submit search on Enter/button and push one URL state. Push status, sort, page and drawer-open changes through Next-integrated native history; useSearchParams then updates without a full page navigation [7]. Reset page atomically when filters/sort change. Keep a transient input draft in sync on Back/Forward. The same mounted table consumes a new query key; use placeholderData: keepPreviousData, aria-busy and an updating indicator [8].

Drawer, cache and conflict implementation

Keep edit sessions distinct from query results

Capture { baseline, draft, dirtyFields } when details first load for a selected ID. Render all editable fields from draft. Background refetches may update the detail cache, but cannot replace draft or baseline.version. A different ID or create mode starts another session; do not key the entire screen/table by URL parameters.

Event	UI behavior	Cache behavior
POST succeeds	Announce creation and close the drawer. The new row may not match the current filters; do not force it into this page.	Set returned detail data; invalidate all company list variants so counts, membership and order recompute.
PATCH succeeds	Announce success and close; discard the completed draft.	Cancel stale detail/list reads before publishing the mutation result. Set the returned detail, then invalidate list prefix. Do not patch a single row into every cached page.
PATCH 409	Keep input and original version; show alert; block save pending reload/review. Never auto retry.	Mark detail stale with refetch disabled and invalidate lists. If optimistic UI is added later it must roll back; this plan waits for the server.
Explicit latest reload	Keep the draft while loading. On failure show Retry. On success show latest values and reapply only dirty fields for review.	Cancel an older detail fetch, then fetchQuery with staleTime=0 using companyOptions(id). The fetched latest record replaces the detail cache.
Reviewed resubmit	Start a new baseline at the fetched version. User explicitly submits reviewed changes; retain original draft snapshot until success or close.	Send if_version from this new baseline. Repeated 409 uses the same path; success uses returned record.
422 / network / 404	422: field/summary errors. Network: retain draft with Retry. Missing detail: not-found state with Close.	Do not publish the failed draft as authoritative data. Keep query failures scoped to the relevant region.

Concrete conflict example

Both tabs open version 1. Tab A changes status and saves, yielding version 2. Tab B edits only name, sends if_version: 1 and receives 409. Its name remains visible. Reload keeps that name, adopts Tab A's untouched status, shows the latest baseline and asks for review. A second explicit save sends if_version: 2, yielding version 3. If both tabs changed name, show the latest name beside the draft so the user makes the decision.

Locale and accessible modal

To retain /companies, store only the selected locale (en/nl) in a cookie. The server layout reads it, dynamically imports that dictionary and passes just its messages and locale to a client provider [9]. Toggle the cookie then router.refresh; keep providers, list and edit session identity stable. Use Intl.DateTimeFormat with en-GB or nl-NL and a consistent timezone such as UTC; use Intl.NumberFormat for counts.

A native dialog opened with showModal can be styled as a side drawer; synchronize its close/cancel events with removal of drawer parameters [11]. On an in-app open, Back can close it; an explicit Close/Escape removes those parameters with replace so direct links never navigate away. Verify Tab/Shift+Tab trapping, initial focus and opener restoration in a real browser. DOM focus/storage effects are acceptable; effect-driven API loading is not.

Test plan and release checks

Required tests should exercise user-visible behavior and actual query integration. Give each component test a fresh QueryClient, disable test retries and mock the network with MSW. Do not mock useQuery into a desired answer or hide production defects with test-only component behavior.

Level	Scenario and meaningful assertions
Required unit	URL normalization/query keys: equivalent default URLs share a key; changing q, status, sort, limit or offset changes it; company/drawer state does not. Alternatively test client bearer header and typed error handling.
Required component: filtering	Render list, submit name/domain search and status; assert URL, actual requested parameters and matching rows. Verify page reset, mounted table identity and Back restoration through a router/history test adapter.
Required component: 409	Open v1, type edits, server returns 409. Assert alert, same input, original if_version and no automatic retry. Reload v2, assert draft kept and untouched fields updated; review/save sends v2; response v3 updates detail and invalidates list.
Focused API checks	40-seed total, 20-item paging, name/domain match, status and four sorts; missing bearer 401 on each operation; missing ID 404; empty name 422; POST 201/v1; PATCH increments and rejects stale version without mutation.
Manual browser smoke	Refresh/share drawer URL; Back/Forward filter and drawer states; create/edit; list/drawer retries and empty/404 states; 422; forced 401 via a test response; NL/EN labels/dates; keyboard trap/Escape/focus return.

Reproduce the conflict without adding a demo feature

Use two tabs that opened the same company at the same version. Save in A, then save a different draft in B. If focus refetch occurs, the form must still retain its captured baseline/version. Follow Reload latest and keep my edits, review, then save. Document these steps in README.

Run on the final revision

```
pnpm install --frozen-lockfile
pnpm lint
pnpm typecheck
pnpm test
pnpm build
pnpm start
```

Check the production build early: development can conceal a missing Suspense boundary around useSearchParams [2]. Next.js 16 uses the ESLint CLI; do not configure pnpm lint as next lint [3]. Ensure typecheck works on a fresh checkout: if generated Next route types are required, run next typegen before tsc or keep source types independent of generated helpers.

A future automated E2E test would cover real routing, direct drawer links, sign-in, keyboard focus and two-tab conflict behavior. Within this scope, the required component/unit suite plus the browser walkthrough is the delivery floor. Record only checks actually performed; passing results must come from the built app.

README and submission checklist

README content to write after implementation

Run: exact supported Node/pnpm versions, install/dev/test/build/start commands, demo sign-in instructions, initial seed behavior and the single-process memory reset limitation.

Decisions and why: URL owns list/drawer state; TanStack Query owns server data; draft state owns the captured version; handwritten typed client and runtime schemas; no optimistic write UI; selected-language dictionary; fake-token storage trade-off; native modal behavior.

How to review: show a sample drawer link, exact two-tab 409 reproduction, filtering/Back behavior and how retry, 422 and unauthorized flows were checked. Report the actual four command results.

More time: real identity integration, persistent transactional storage, real generated client and API-enforced tenant scope, plus automated browser coverage for focus/routing/two tabs. These are discussion points, not features to add now.

Conscious omissions: no real auth/token refresh, persistence, organization switching, advanced domain validation, automatic conflict merge, draft recovery after deliberate drawer close, optional E2E automation or visual extras. Revise this list to match what was actually built.

Suggested meaningful commit sequence

1. Scaffold strict Next.js app and quality scripts.
2. Add seeded company API with validation and version checks.
3. Add typed transport, demo auth and query options.
4. Add URL-driven company list and filtering test.
5. Add create/edit drawer and conflict recovery test.
6. Finish locale, accessibility, verification and README.

Commit completed increments as work progresses. Submit a public repository or explicitly share private access. Put the nine short responses in ANSWERS.md or link their submitted location from README. A live deployment is not requested and complicates the memory-store limitation.

Final self-review

All nine written answers are present and personalized. All PRD F01-F10 and T01-T05 checks have evidence. README covers all four requested topics. Git contains meaningful commits and the lockfile. The final revision passes all four required commands. Remove debugging, unused files, secrets, any and @ts-ignore. Reproduce the conflict once more in the production build.

Be ready to explain: why a refreshed cache must not reset a draft; why the saved version belongs to an edit session; why list invalidation is safer than blindly patching cached pages; why a 401 cannot retry indefinitely; and which production guarantees the mock intentionally does not offer.

Technical references

The assessment PDF is the scope authority. These official references support the proposed implementation choices; the plan is an engineering synthesis. Check installed package versions and matching docs before coding.

[1] Server and Client Components
[2] useSearchParams and Suspense
[3] Next.js 16 upgrade changes
[4] Route Handler params
[5] TanStack Query options
[6] TanStack Query keys
[7] Native History API
[8] Paginated queries
[9] Internationalization
[10] Auth0 token storage
[11] MDN dialog behavior

References checked 9 November 2026.